🎓 جزوه طلایی سیستم‌عامل

فصل ۱۰: حافظه مجازی - بخش نهم (ملاحظات نهایی و پایان فصل)

۱. پیش‌صفحه‌بندی (Prepaging)

در «صفحه‌بندی درخواستی محض»، وقتی برنامه‌ای شروع می‌شود، تعداد زیادی **خطای صفحه** پشت سر هم رخ می‌دهد تا صفحات اولیه بارگذاری شوند.

- **ایده:** بیاییم تمام (یا بخشی از) صفحات مورد نیاز را **یکجا** و قبل از شروع اجرا به حافظه بیاوریم.
- **سوال مهم:** آیا می‌صرفد؟
  - اگر صفحات آورده شده استفاده شوند: **بله** (چون سربار I/O کم می‌شود).
  - اگر صفحات آورده شده استفاده نشوند: **خیر** (حافظه و پهنای باند دیسک هدر رفته).
- **نتیجه:** اگر کسر کوچکی از صفحات پیش‌خوانی شده استفاده شوند، بهتر است از همان روش "درخواستی" استفاده کنیم.

۲. اندازه صفحه (Page Size)

انتخاب اندازه صفحه (مثلاً KB۴ یا MB۴) یک تصمیم حیاتی در طراحی سیستم‌عامل است. هیچ اندازه‌ای "بهترین" نیست، بلکه مصالحه (Trade-off) است:

صفحه بزرگ	صفحه کوچک	ویژگی
❌ بیشتر	✅ کمتر (هدررفت کمتر)	**تکه‌تکه شدن داخلی (Internal Fragmentation)**
✅ کوچکتر	❌ بزرگتر (نیاز به حافظه بیشتر)	**اندازه جدول صفحه (Page Table Size)**
✅ سریعتر (سربار کمتر)	❌ کندتر (سربار زیاد برای دیتای کم)	**سرعت I/O (دیسک)**
❌ ممکن است دیتای اضافی بیاید	✅ دقیق‌تر (فقط دیتای لازم می‌آید)	**همسایگی (Locality)**

✅ کمتر ❌ بیشتر **تعداد خطای صفحه**

روند تاریخی: با ارزان شدن حافظه و سریع‌تر شدن CPUها، اندازه صفحات به سمت **بزرگتر شدن** می‌رود (اکنون KB۴ استاندارد است، اما MB۲ و GB۱ هم رایج شده‌اند).

۳. حجم پوششی TLB (TLB Reach)

- **تعریف:** مقدار حافظه‌ای که مستقیماً از طریق TLB قابل دسترسی است (بدون مراجعه به جدول صفحه در رم).
- $$\text{TLB Reach} = (\text{تعداد ورودی‌های TLB}) \times (\text{اندازه صفحه})$$
- **هدف:** حجم پوششی باید کل **مجموعه کاری (Working Set)** پردازه را پوشش دهد.
- **راهکار افزایش Reach:**
  1. افزایش تعداد ورودی‌های TLB (گران و پرمصرف انرژی).
  2. **افزایش اندازه صفحه** (راهکار موثرتر).
     - *نکته:* سیستم‌های مدرن (مثل لینوکس و ARMv8) از **چند اندازه صفحه** پشتیبانی می‌کنند (مثلاً KB۴ برای کارهای عادی و MB۲ برای برنامه‌های سنگین).

۴. جدول صفحه وارونه (Inverted Page Table)

- **مشکل:** این جداول اطلاعات کامل منطقی (Logical) را ندارند، فقط اطلاعات فیزیکی دارند.
- **چالش در صفحه‌بندی درخواستی:** وقتی خطای صفحه رخ می‌دهد، سیستم باید بداند صفحه مجازی کجای دیسک است. چون جدول وارونه این را ندارد، نیاز به یک **جدول صفحه خارجی (External Page Table)** داریم.
- **نتیجه:** مدیریت پیچیده‌تر می‌شود (ممکن است خودِ خواندنِ جدول خارجی هم خطای صفحه بدهد!).

۵. ساختار برنامه (Program Structure)

آیا برنامه‌نویس می‌تواند روی کارایی حافظه مجازی تاثیر بگذارد؟ **بله، بسیار زیاد!**

مثال معروف آرایه (C Row-Major):

فرض کنید صفحات ۱۲۸ کلمه‌ای داریم و آرایه data[128][128] تعریف شده (هر سطر دقیقاً یک صفحه است).

حالت ۱ (خوب): دسترسی سطری

```
(++for (j = 0; j < 128; j

(++for (i = 0; i < 128; i

;0 = [data[j][i
```

- چون زبان C آرایه‌ها را **سطری** ذخیره می‌کند، تمام کلمات یک صفحه پشت سر هم خوانده می‌شوند.
- **تعداد خطای صفحه:** ۱۲۸ (یکی برای هر صفحه).

حالت ۲ (فاجعه): دسترسی ستونی

```
(++for (i = 0; i < 128; i

(++for (j = 0; j < 128; j

;0 = [data[j][i
```

- اینجا ستونی می‌خوانیم (پرش بین سطرها). یعنی برای هر کلمه به یک صفحه جدید می‌رویم.
- **تعداد خطای صفحه:** ۱۲۸ × ۱۲۸ = **۱۶,۳۸۴** !
- **نتیجه:** کد یکسان، ولی دومی صدها برابر کندتر است!

ساختمان داده‌ها:

- **پشته (Stack):** همسایگی عالی (چون خطی است).
- **جدول درهم‌سازی (Hash Table):** همسایگی ضعیف (چون پخش است).

🔒 ۶. قفل I/O (I/O Interlock)

یک مشکل خطرناک در چندبرنامگی!

سناریو:

1. پردازه A درخواست I/O می‌دهد (مثلاً خواندن از دیسک به بافر خودش در رم).
2. پردازه A منتظر می‌ماند و CPU به پردازه B می‌رسد.

3. پردازه B خطای صفحه می‌دهد و رم کم می‌آید.
4. الگوریتم جایگزینی (Global)، صفحه متعلق به A (که منتظر I/O است) را قربانی می‌کند و بیرون می‌اندازد!
5. **فاجعه:** دیسک کارش تمام می‌شود و دیتا را در آن قاب می‌نویسد. اما آن قاب الان مال پردازه B است! دیتای B خراب می‌شود.

راهکار: قفل کردن (Pinning / Locking)

- صفحاتی که درگیر I/O هستند باید در حافظه **قفل (Lock)** شوند تا الگوریتم جایگزینی نتواند آن‌ها را انتخاب کند.
- پس از اتمام I/O، قفل باز می‌شود.

📝 خلاصه کلیدی فصل ۱۰ (پایان):

1. **Prepaging:** آوردن پیش‌دستانه صفحات (سودمند اگر درست حدس بزنیم).
2. **Page Size:** بزرگتر = I/O سریعتر و جدول کوچکتر، اما هدررفت (Fragmentation) بیشتر.
3. **Program Structure:** برنامه‌نویس با دسترسی حافظه منطبق بر ذخیره‌سازی (Locality) می‌تواند سرعت را صدها برابر کند.
4. **I/O Interlock:** صفحات درگیر I/O نباید Swap شوند (باید Lock شوند).